离散化
Graph

离散化

```cpp
template<class T>
struct discrete
{
    std::vector<T>a;
    int len;
    discrete(std::vector<T> arr,int st = 0)
    {
        std::sort(arr.begin() + st,arr.end());
        arr.erase(std::unique(arr.begin() + st,arr.end()),arr.end());
        a.assign(arr.begin() + st, arr.end());
        len = a.size();
    }
    int size(){return len;}
    //第几小
    int operator[](T x){
        int pos = std::lower_bound(a.begin(),a.end(),x) - a.begin();
        return pos + 1;
    }
};
```

Graph

```cpp
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };
    std::vector<edge>e;
    std::vector<int>heads;
    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }
    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
//example
void solve()
```

```cpp
{
    int n,m;
    std::cin >> n >> m;
    Graph g(n,m);
    for(int i = 1;i <= m;i++)
    {
        int u,v;
        std::cin >> u >> v;
        g.add(u,v);
    }
    for(int u = 1;u <= n;u++)
    {
        for(int i = g.heads[u];~i;i = g.e[i].nx)
        {
            int v = g.e[i].to;
        }
    }
}
```